Algorithmes d’apprentissage non supervisé

01-07

Méthodes avancées

@ 2026 MKL.AI - Tous droits réservés.

Au programme

- Algorithme BFR
- Algorithme CURE
- Implémentation MapReduce (K-means)
- Lectures et références

@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR

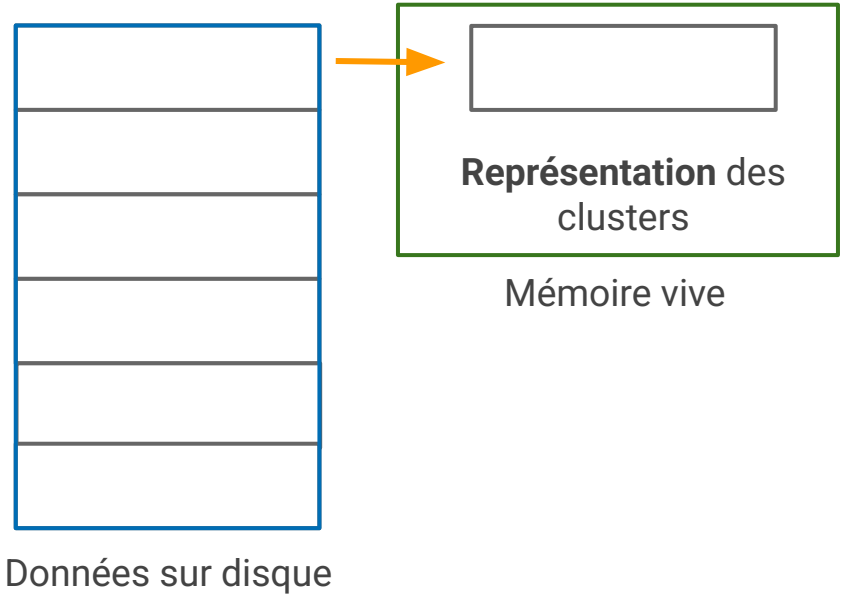
Algorithme BFR

- **BFR ➜** Bradley - Fayyad - Reina
- L’**algorithme BFR** est une extension de l’algorithme des K-moyennes appliqué aux données **volumineuses** et en **haute dimension**
- Applicable aux **espaces euclidiens** seulement
- Cet algorithme permet de notamment réaliser le clustering en **une seule lecture des données** !

@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR

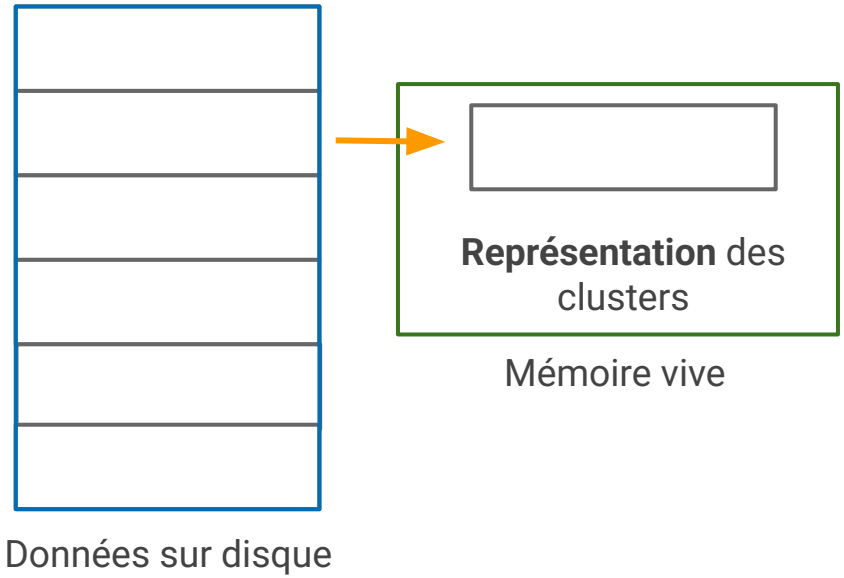
- Données “volumineuse” ⇒ ne tiennent pas en **mémoire vive**



@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR

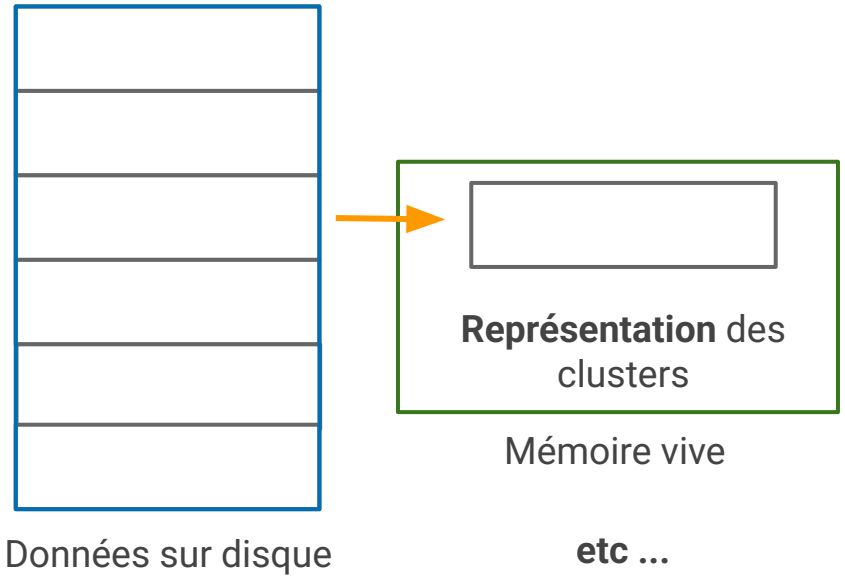
- Données “volumineuse” ⇒ ne tiennent pas en **mémoire vive**



@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR

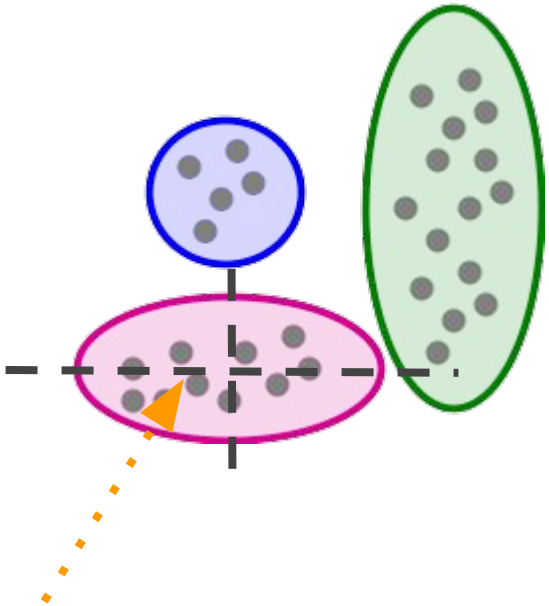
- Données “volumineuse” ⇒ ne tiennent pas en **mémoire vive**



@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR

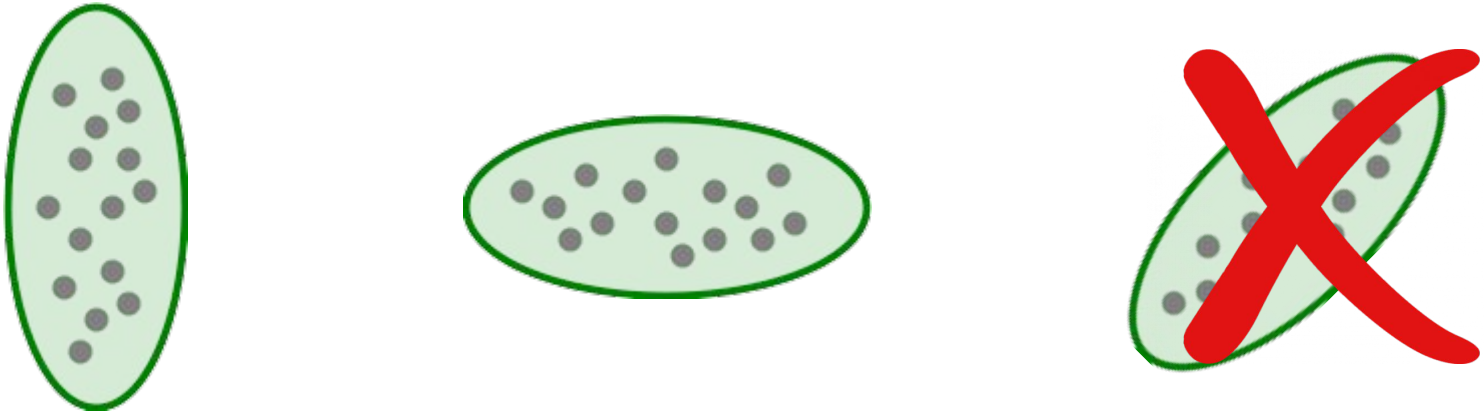
- On suppose que les clusters sont **normalement distribués** autour de leur centroïdes
- La moyenne et l'écart-type pour un cluster peut varier selon les dimensions, mais celles-ci doivent être **indépendantes**



@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR

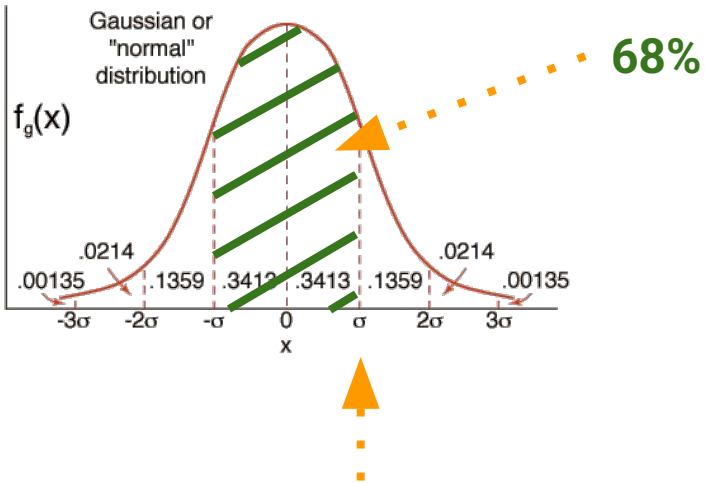
- **Les dimensions sont indépendantes** ➜ les clusters forment des ellipses alignées sur les axes



@ 2026 MKL.AI - Tous droits réservés.

Distribution normale

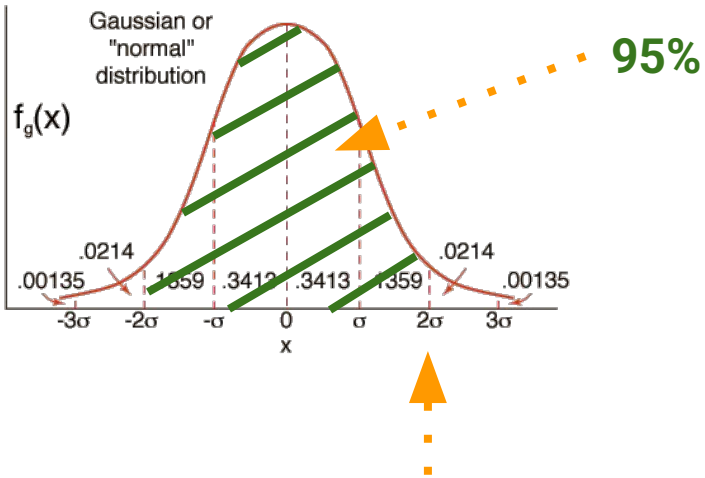
- Permet de quantifier la **probabilité** de trouver un point au sein d'un cluster à une distance donnée du centroïde selon chaque dimension



@ 2026 MKL.AI - Tous droits réservés.

Distribution normale

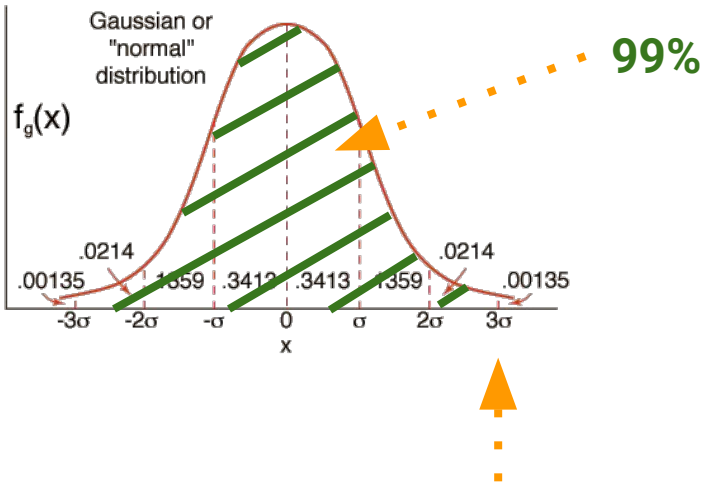
- Permet de quantifier la **probabilité** de trouver un point au sein d'un cluster à une distance donnée du centroïde selon chaque dimension



@ 2026 MKL.AI - Tous droits réservés.

Distribution normale

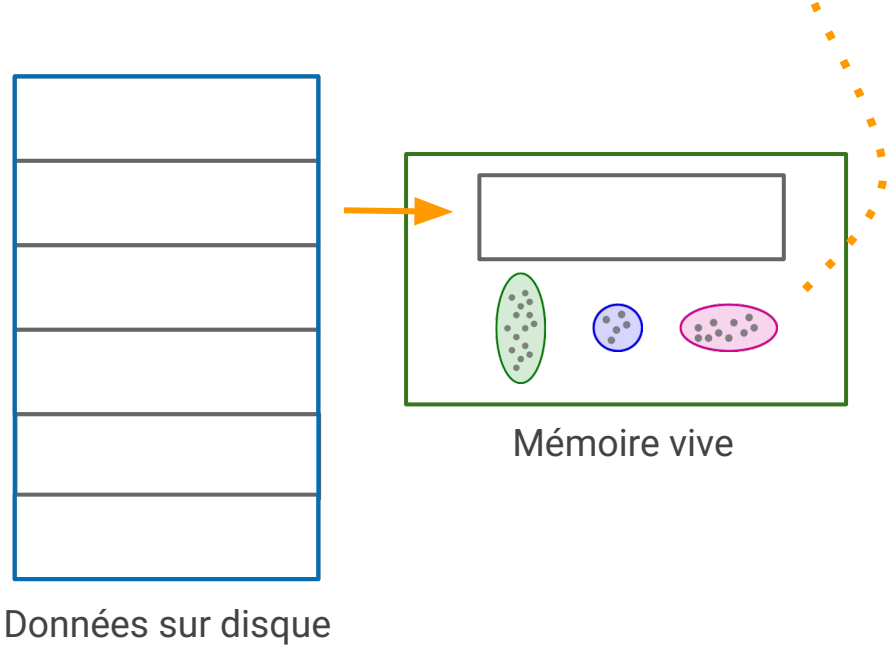
- Permet de quantifier la **probabilité** de trouver un point au sein d'un cluster à une distance donnée du centroïde selon chaque dimension



@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR - Représentations des clusters

- Les points (observations) sont lues par morceaux depuis le stockage disque, puis représentées par leurs **statistiques (ici des métadonnées)**



@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR - Sélection des k centroïdes initiaux

- L’étape initiale, correspondant au premier chargement des données, exécute l’**algorithme des K-moyennes**. L’approche est ici légèrement différente des celle vue au chapitre 01-02
  - Choisir k points les plus éloignés les uns des autres que possible
  - Exécuter l’algorithme des K-moyennes sur un échantillon des données
  - Pour chaque cluster, choisir le point le plus proches des centroïdes

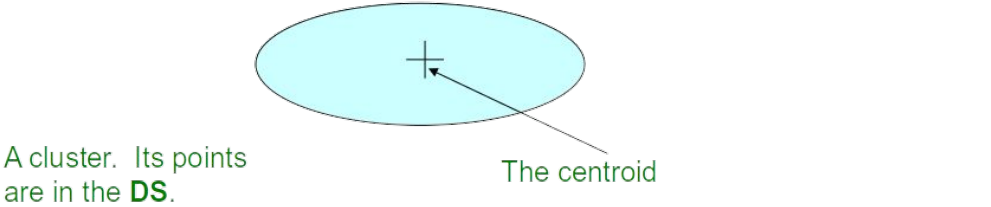
@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR - 3 ensembles

- **Discard set (DS)**
  Observations suffisamment proches d’un centroïde “représentées” par les méta-données
- **Compression set (CS)**
  Groupe d’observation suffisamment proches entre elles, mais pas proche d’un centroïde existant. Ces points sont représentés par leur métadonnées sans êtres assignés à un cluster
- **Retained set (RS)**
  Point isolés en attente d’assignation à un CS. Ce sont les seuls points à être gardés en mémoire

@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR - Discard Set (DS)

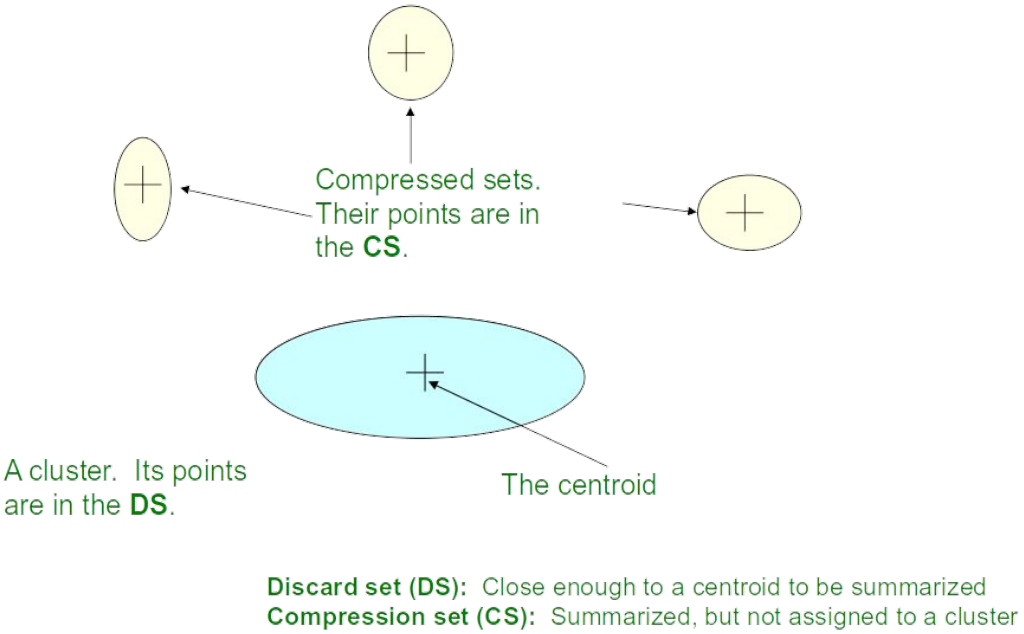


Discard set (DS): Close enough to a centroid to be summarized

Extrait de [1] - Représentation “Galaxies”

@ 2026 MKL.AI - Tous droits réservés.

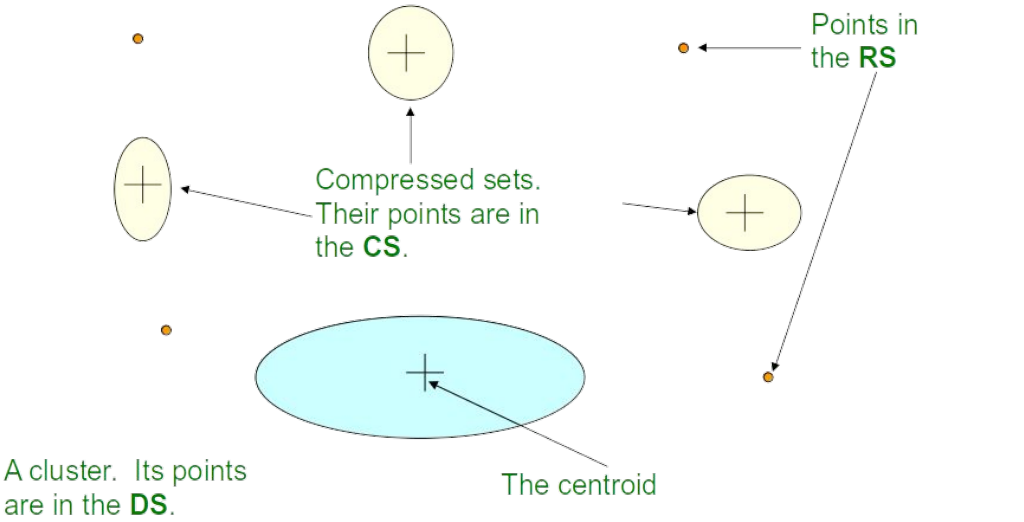
Algorithme BFR - Compression Set (CS)



Extrait de [1] - Représentation “Galaxies”

@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR - Retained Set (RS)



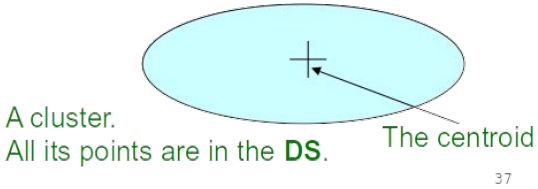
Discard set (DS): Close enough to a centroid to be summarized
Compression set (CS): Summarized, but not assigned to a cluster
Retained set (RS): Isolated points

Extrait de [1] - Représentation “Galaxies”

@ 2026 MKL.AI - Tous droits réservés.

Représentation des ensembles de points (DS)

- Pour chaque cluster, le DS est représenté par
  - **Le nombre de points N**
  - **Le vecteur SUM**
    Le $i^{ième}$ élément est la somme des coordonnées des points dans la $i^{ième}$ dimension
  - **Le vecteur SUMSQ**
    Le $i^{ième}$ élément est la somme des carrés des points dans la $i^{ième}$ dimension



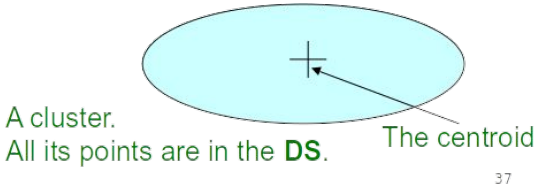
@ 2026 MKL.AI - Tous droits réservés.

Représentation des ensembles de points (DS)

- Si d est le nombre de dimensions, chaque cluster est représenté par $2d$ + 1 valeurs

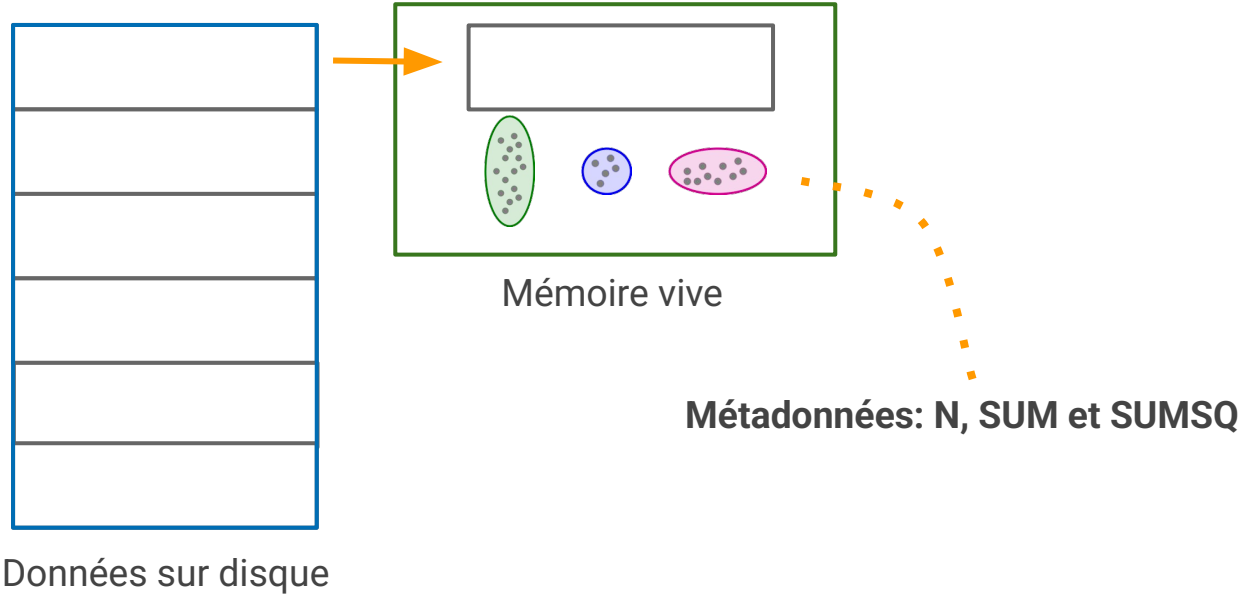
- La moyenne sur la dimension **i** sur chaque dimension (et donc le centroïde !) peut être obtenue par **SUM_i / N**

- De même, la variance d'un cluster sur la dimension **i** est **($SUMSQ_i$ / N) – (SUM_i / N)^2**



@ 2026 MKL.AI - Tous droits réservés.

Algorithme BFR - Retained Set (RS)



@ 2026 MKL.AI - Tous droits réservés.

Clustering (1/2)

Pour les points chargés en mémoire:

1. Trouver les points **suffisamment proches** d'un centroïde et ajouter ces points au cluster et au Discard Set

   Ces points étant proches d'un centroïde peuvent être "représentés" et ensuite retirés (discarded)

1. Utiliser un **algorithme de clustering** (K-moyennes, hiérarchique, …) sur l'ensemble des points restants et sur ceux du Retained Set

@ 2026 MKL.AI - Tous droits réservés.

Clustering (2/2)

3. Discard Set: ajuster les représentations des clusters pour prendre en compte les nouveaux points

   ⇒ Ajouter **Ns**, **SUMs** et **SUMSQs**

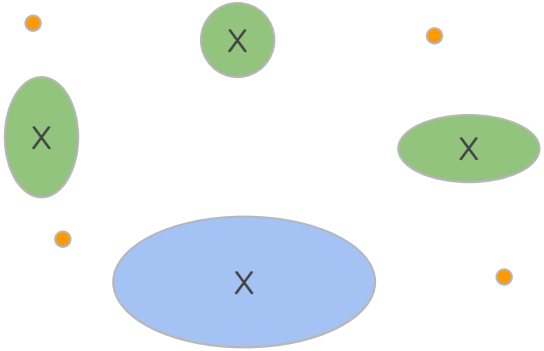
4. Considérer la fusion des CS

5. Si cette étape est la dernière (dernier segment de points), fusionner les CS et tous les points de RS aux clusters les plus proches

@ 2026 MKL.AI - Tous droits réservés.

Quelques questions restent non résolues ...

1. Comment décider si un point est **suffisamment proche** d'un cluster de sorte à l'y ajouter ?
2. Comment décider de combiner ou non deux sous-clusters du Compression Sets ?



Représentation des ensembles DS, CS et RS

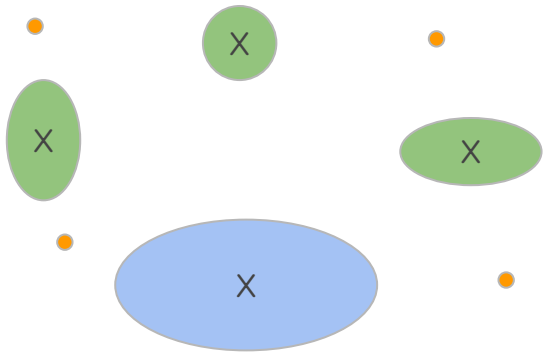
@ 2026 MKL.AI - Tous droits réservés.

Quelques questions restent non résolues ...

1. Comment décider si un point est **suffisamment proche** d'un cluster de sorte à l'y ajouter ?

L'algorithme BFR suggère deux façons:

- Lorsque la **distance de Mahalanobis** est inférieure à un certain seuil
- Lorsque la **probabilité** que le point appartienne au centroïde le plus proche est suffisamment élevée



Représentation des ensembles DS, CS et RS

@ 2026 MKL.AI - Tous droits réservés.

Distance de Mahalanobis

- La distance de Mahalanobis est la **distance au centroïde normalisée**
- Considérons un cluster **C** de centroïde (c_1, ..., c_d) et d'écart-type (σ_1, ..., σ_d) ainsi qu'un point **P** (x_1, ..., x_d)
- Alors la **distance normalisée** entre **P** et **C** selon la dimension **i** s'exprime sous la forme:

$$y_i = \frac{(x_i - c_i)}{\sigma_i}$$

- La distance de Mahalanobis du point P au cluster C est:

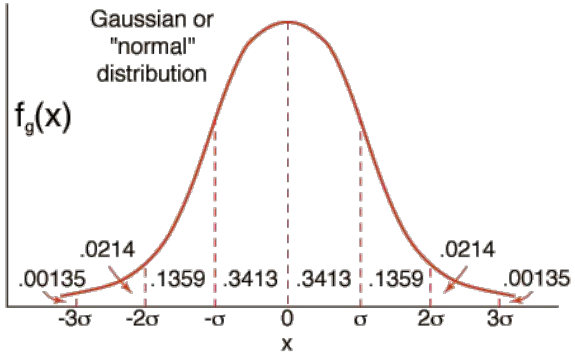
$$\mathrm{MD} = \sqrt{\sum_{i=1}^{d} y_i^2}$$

@ 2026 MKL.AI - Tous droits réservés.

Critère d'acceptance de Mahalanobis

- Supposons que le point **P** soit situé à un écart-type du centroïde sur chaque dimension
- La distance de Mahalanobis (MD) est alors

$$\sqrt{d}$$



68% des points ont $\mathrm{MD} \leq \sqrt{d}$

95% des points ont $\mathrm{MD} \leq 2\sqrt{d}$

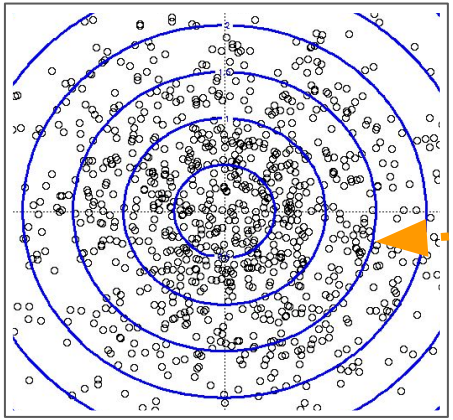
99% des points ont $\mathrm{MD} \leq 3\sqrt{d}$

exemple

➜ On intègre le point **P** au cluster **C** si sa MD est inférieure à un certain seuil

@ 2026 MKL.AI - Tous droits réservés.

Comparaison dist. de mahalanobis et euclidienne

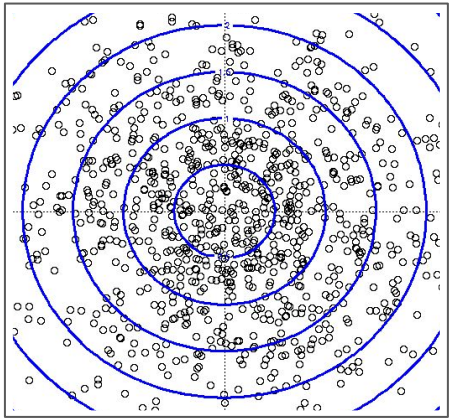


Point **uniformément** distribués
Distance euclidienne

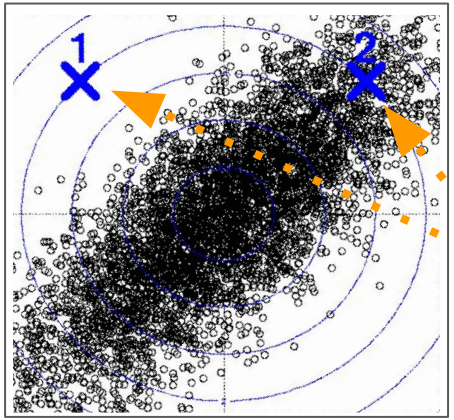
Les lignes de contour représentent les points **équidistant** de l'origine

@ 2026 MKL.AI - Tous droits réservés.

Comparaison dist. de mahalanobis et euclidienne



Point **uniformément** distribués
Distance euclidienne

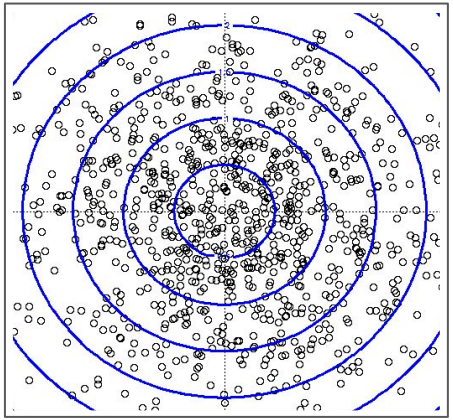


Point **normalement** distribués
Distance euclidienne

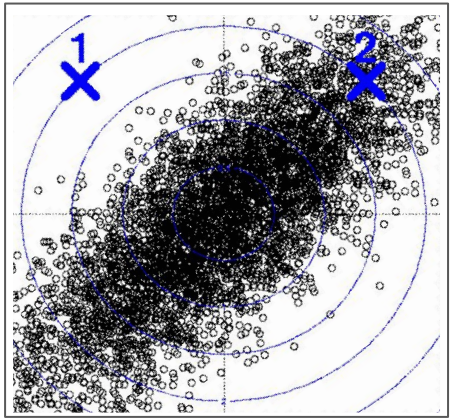
Ici les points 1 et 2 sont à une même distance euclidienne de l'origine

@ 2026 MKL.AI - Tous droits réservés.

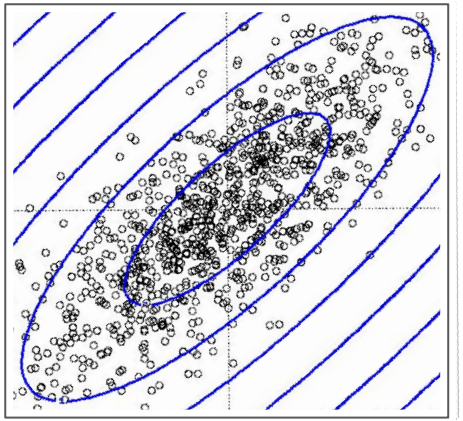
Comparaison dist. de mahalanobis et euclidienne



Point **uniformément** distribués
Distance euclidienne



Point **normalement** distribués
Distance euclidienne

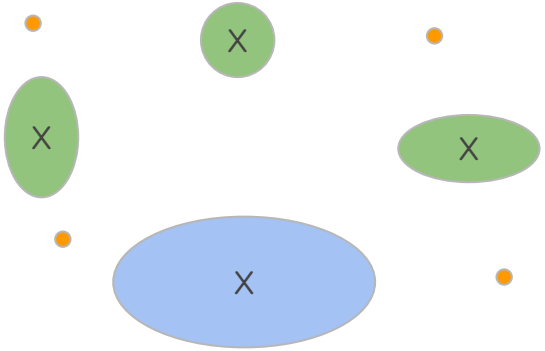


Point **normalement** distribués
Distance de **Mahalanobis**

@ 2026 MKL.AI - Tous droits réservés.

Quelques questions restent non résolues ...

2. Comment décider de combiner ou non deux sous-clusters du Compression Sets ?



Représentation des ensembles DS, CS et RS

Calculer la variance des deux sous-clusters **combinés**

➜ N, SUM, et SUMSQ permettent de réaliser ce calcul très rapidement !

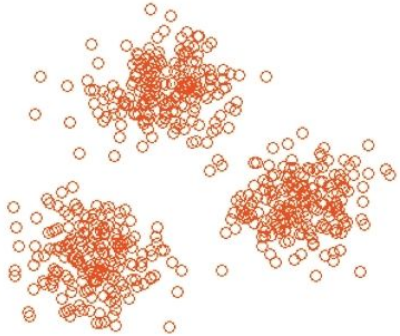
Combiner les deux sous-clusters si la variance calculée précédemment est **inférieure à un certain seuil**

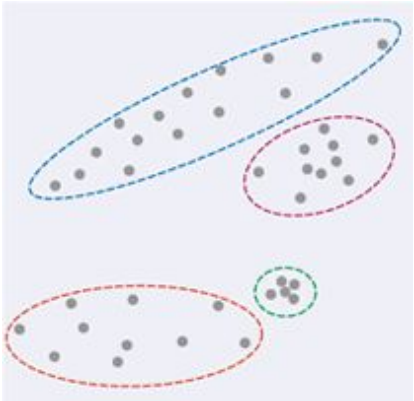
@ 2026 MKL.AI - Tous droits réservés.

Algorithme CURE

Limitations des K-moyennes et de BFR

- Suppose que les clusters soient **normalement distribués** sur chaque dimension
- “Axes” des clusters doivent être parallèles aux axes des dimensions
- Les ellipses ayant un angle et les formes non globulaires ne sont pas prises en compte

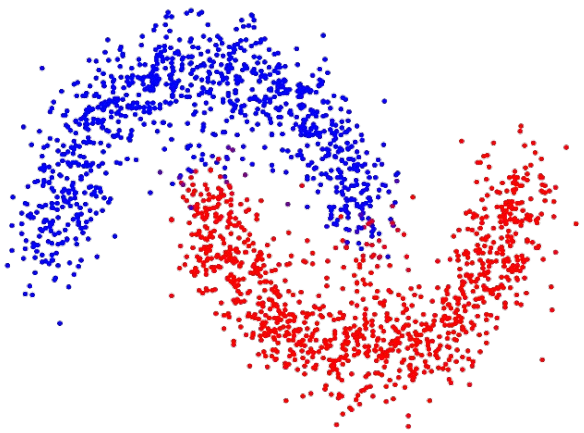




@ 2026 MKL.AI - Tous droits réservés.

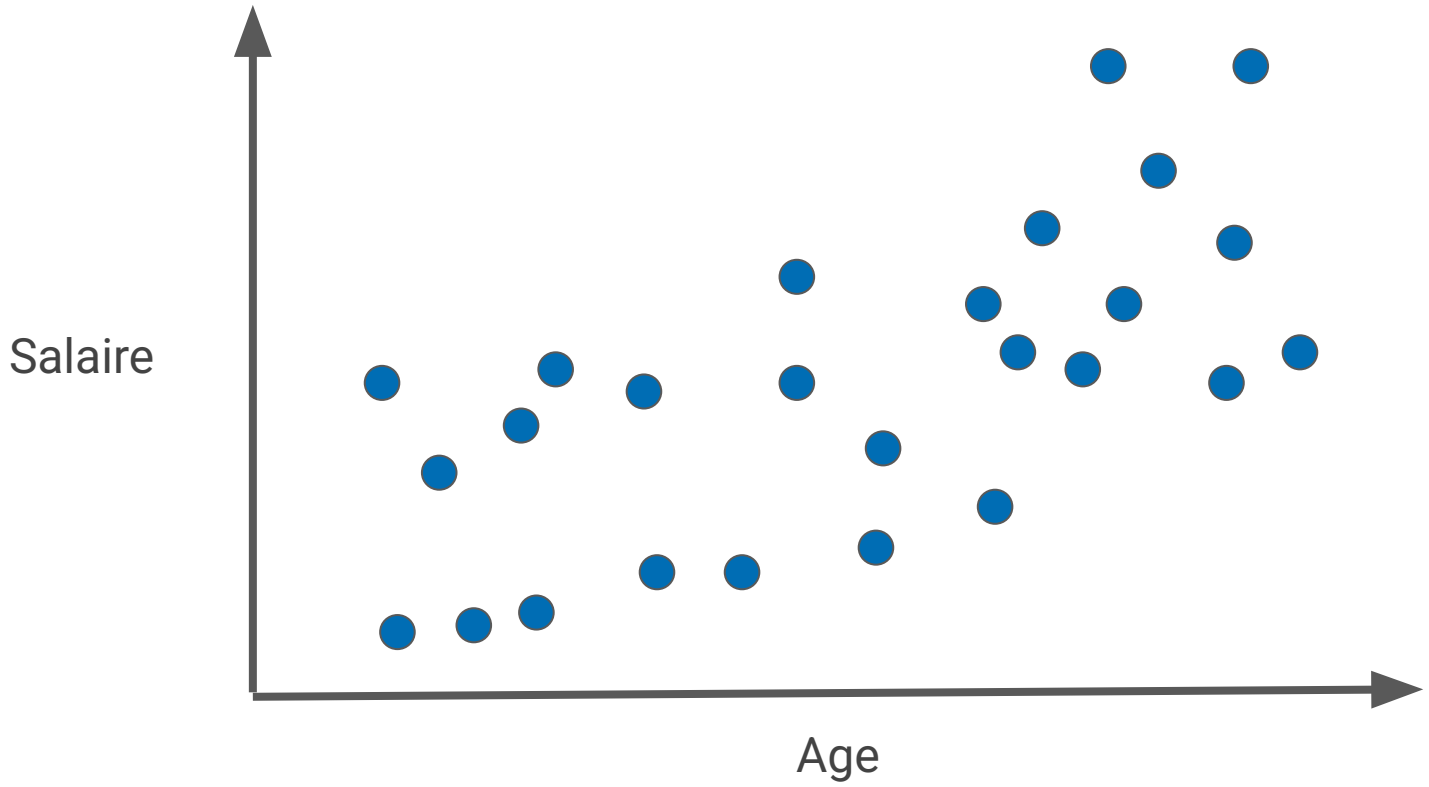
Algorithme CURE

- **CURE** (Clustering Using REpresentatives)
- Suppose une **distance euclidienne**
- Les clusters peuvent avoir n'importe quelle forme et distribution
- Se base sur une collection de **représentants (points représentatifs)**



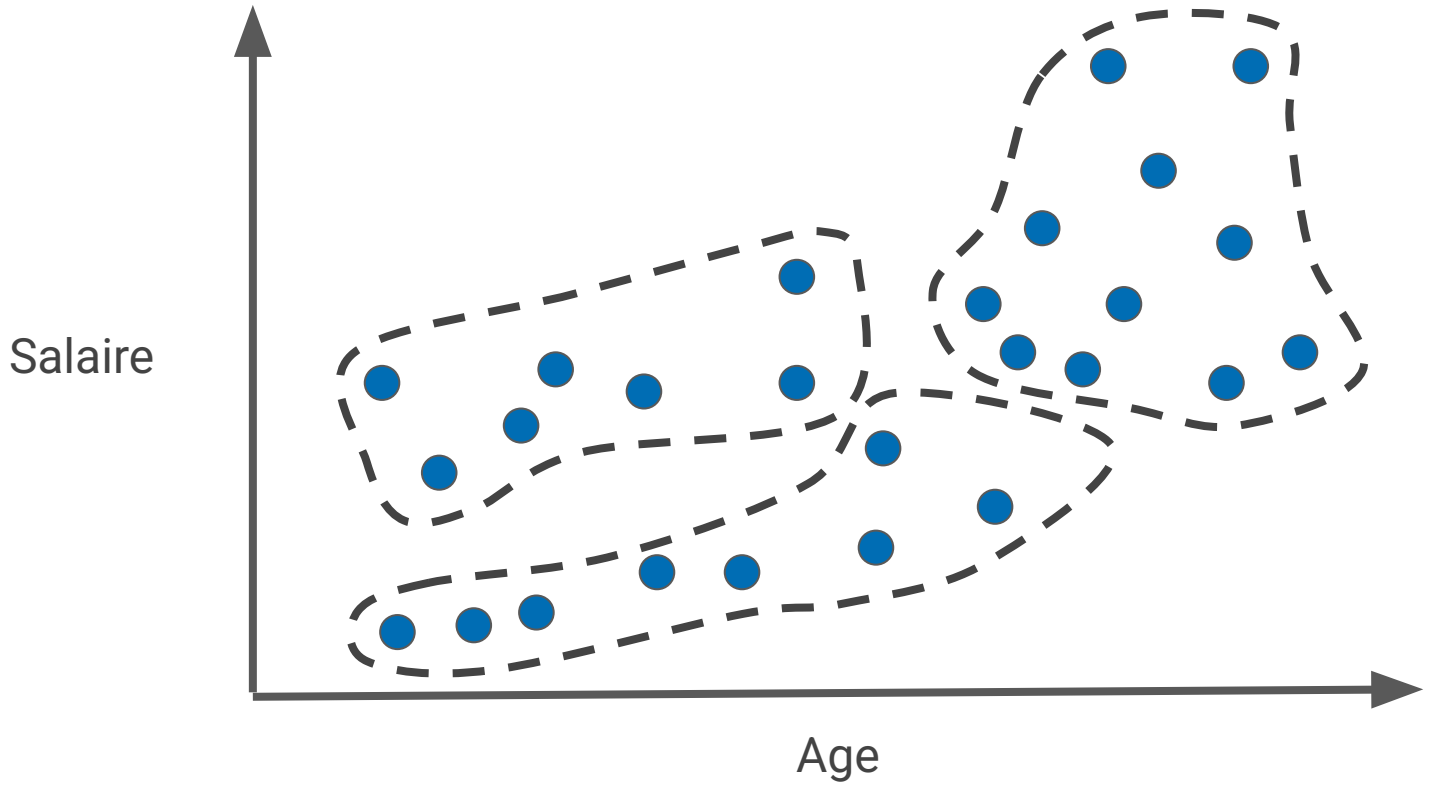
@ 2026 MKL.AI - Tous droits réservés.

Exemple: salaires des enseignants d’une université



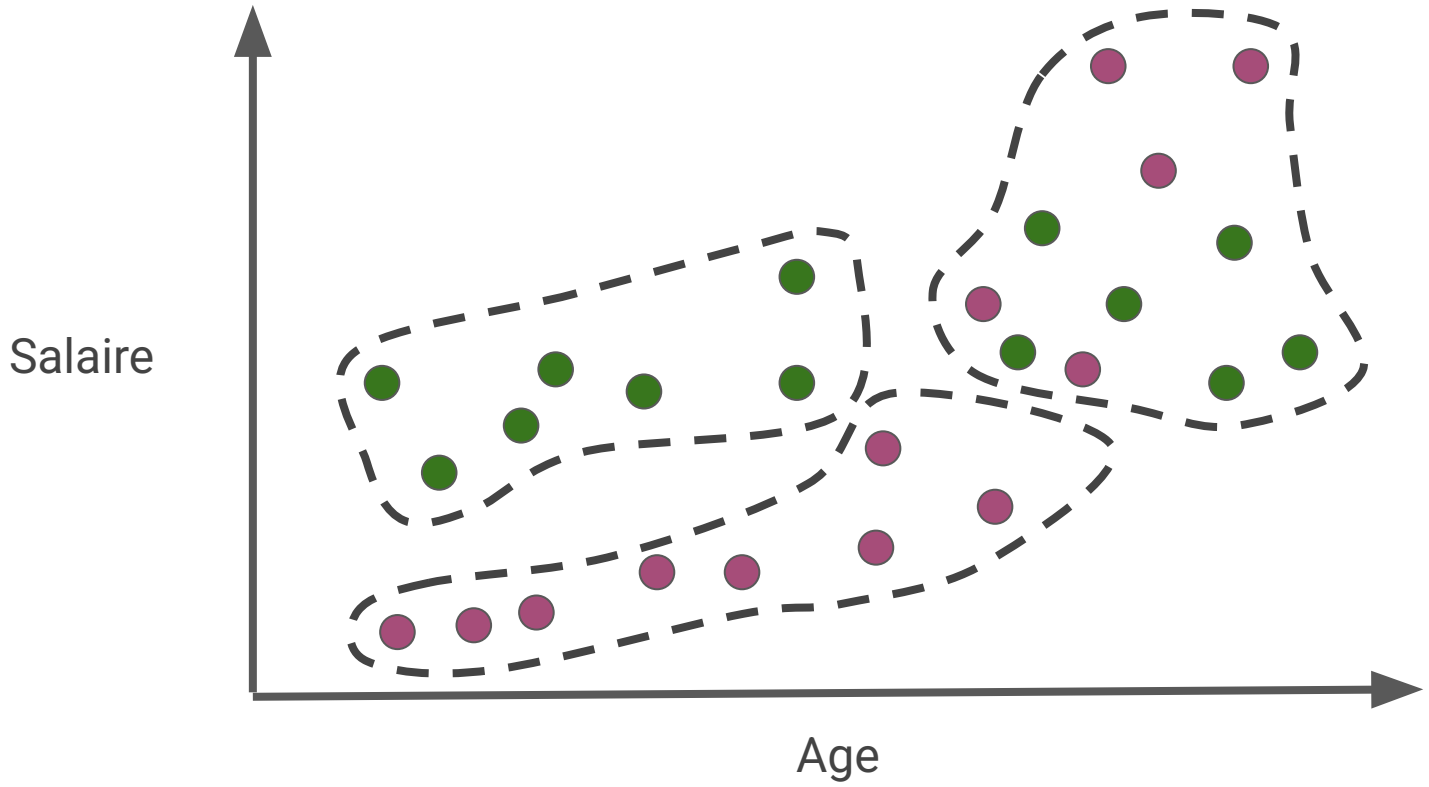
@ 2026 MKL.AI - Tous droits réservés.

Exemple: salaires des enseignants d’une université



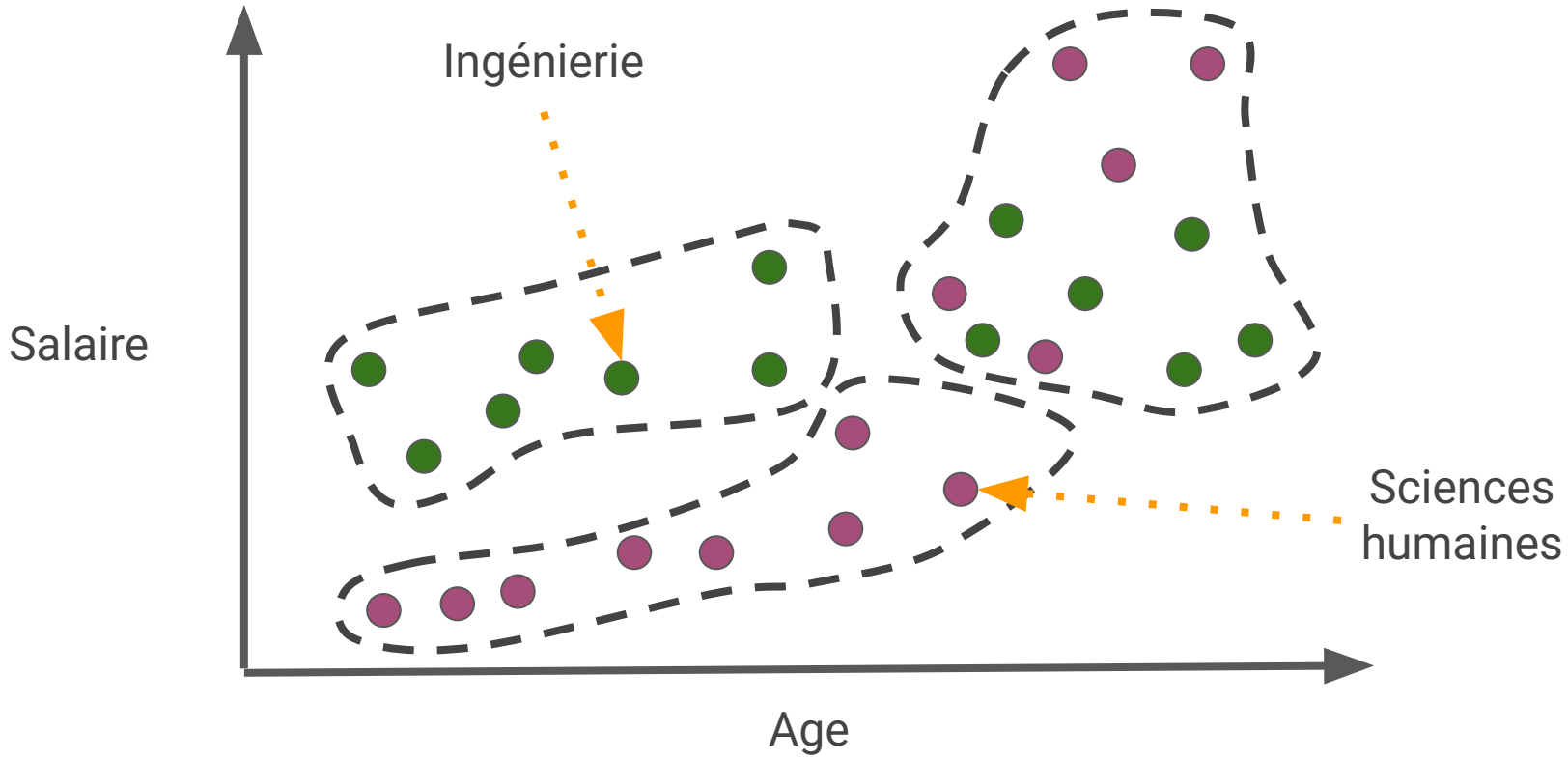
@ 2026 MKL.AI - Tous droits réservés.

Exemple: salaires des enseignants d’une université



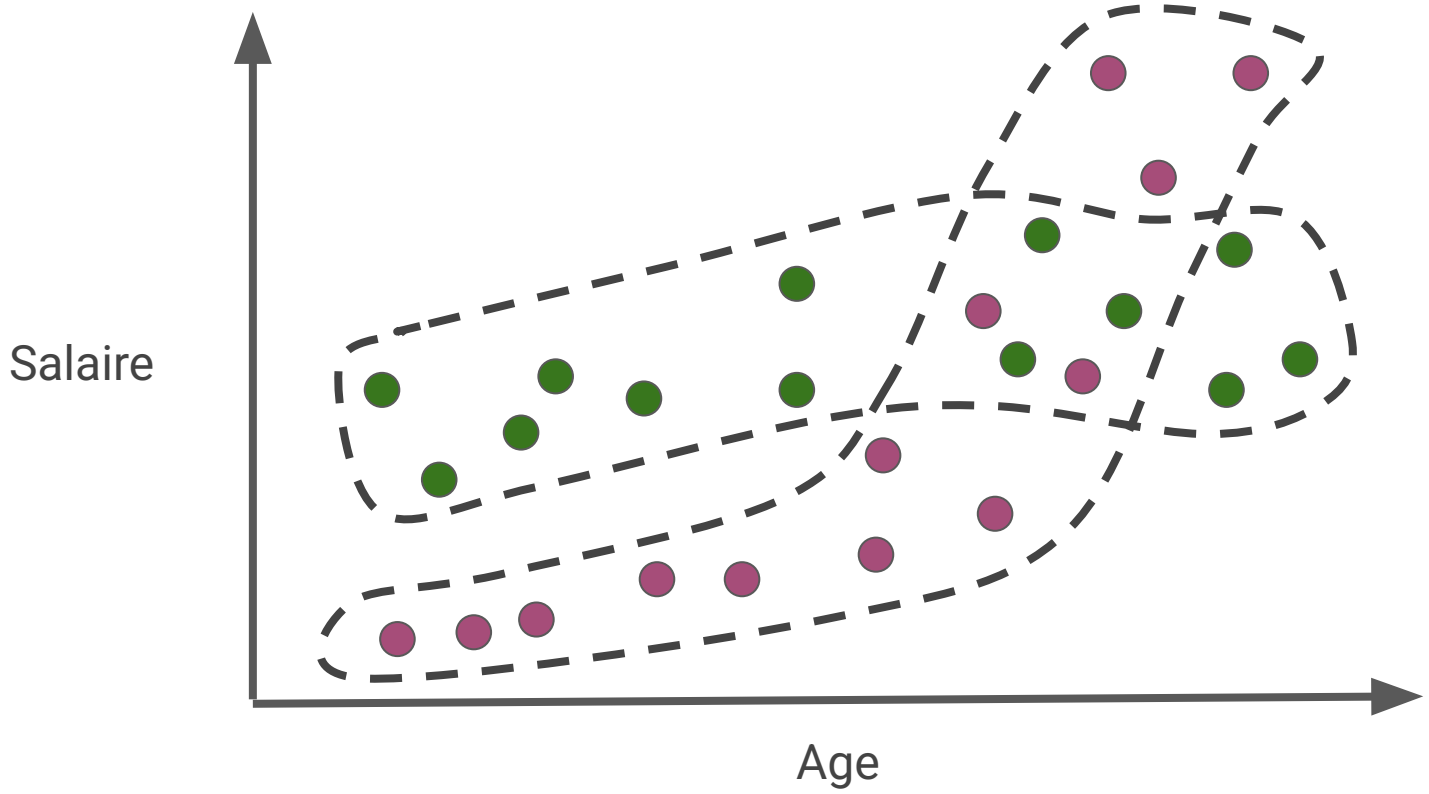
@ 2026 MKL.AI - Tous droits réservés.

Exemple: salaires des enseignants d'une université



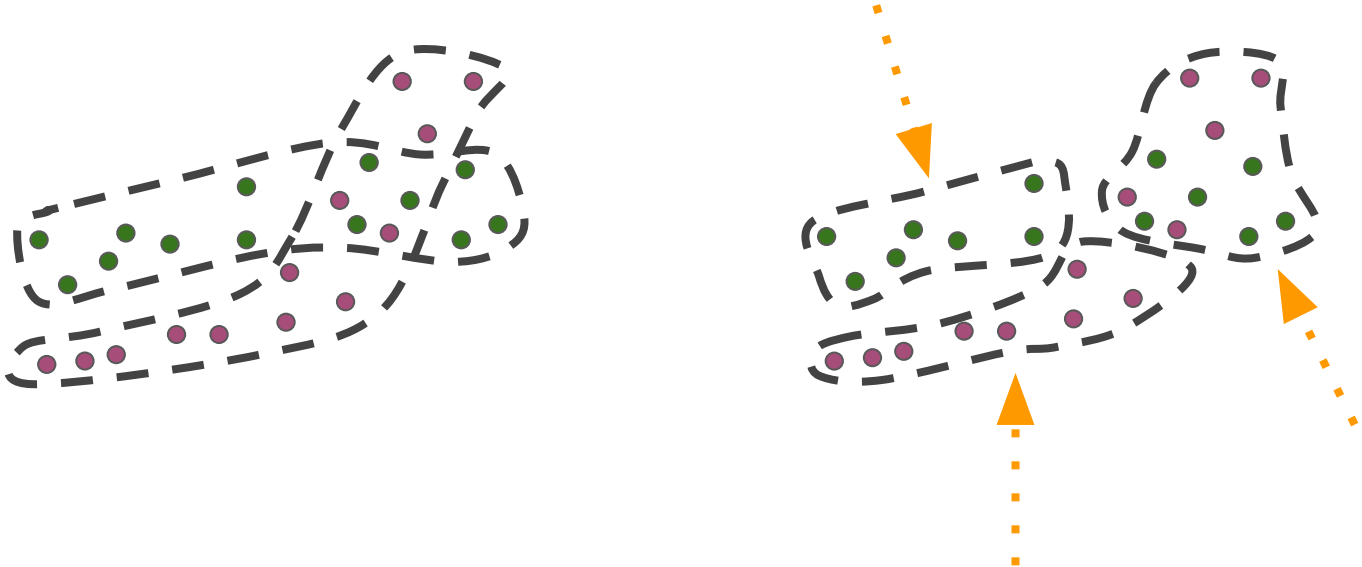
@ 2026 MKL.AI - Tous droits réservés.

Exemple: salaires des enseignants d’une université



@ 2026 MKL.AI - Tous droits réservés.

Exemple: salaires des enseignants d’une université



@ 2026 MKL.AI - Tous droits réservés.

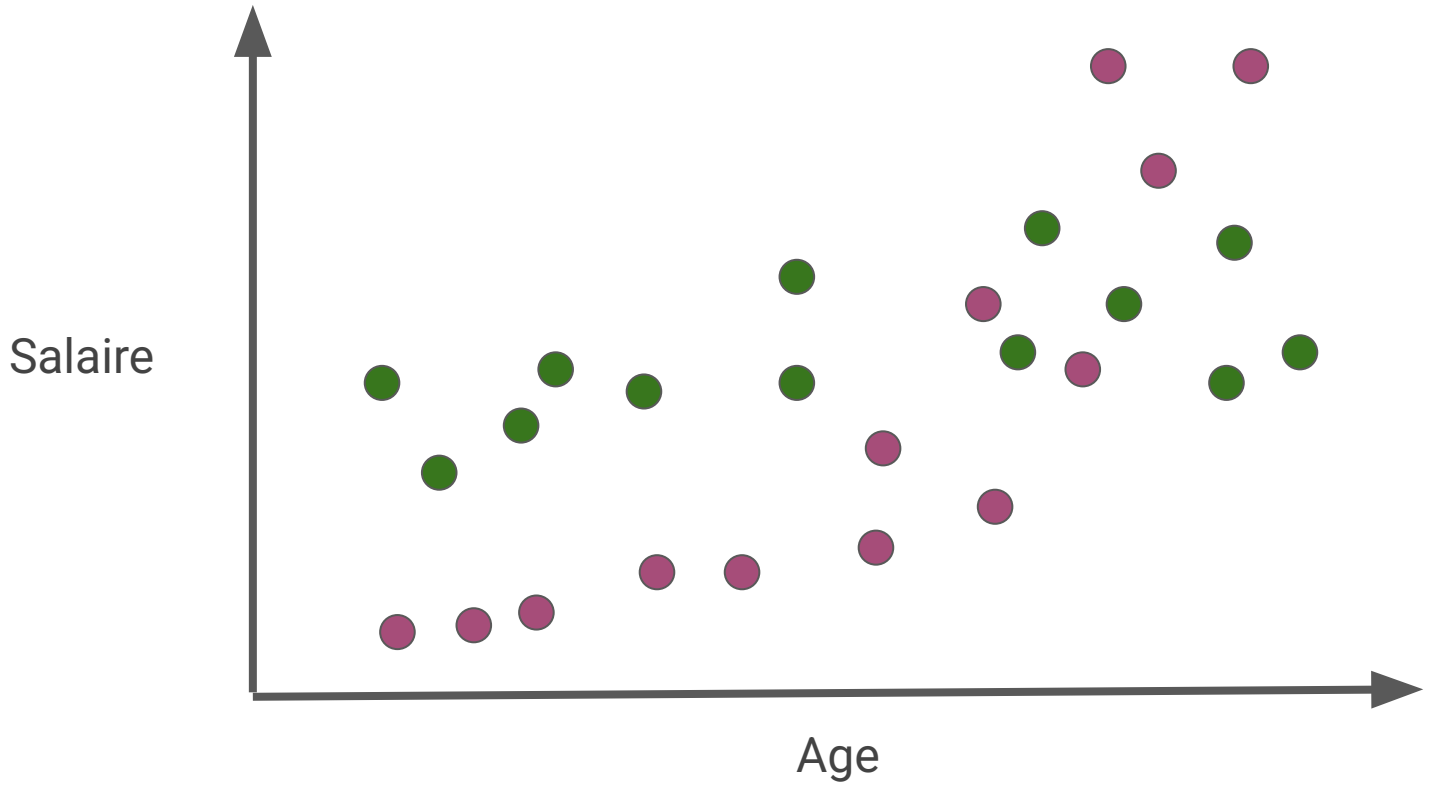
Algorithme CURE (1/2)

- L’algorithme **CURE** est composé de **deux séquences de lecture** des données
- **Séquence 1**

  0. **Choisir au hasard un échantillon de points**
     Ces points doivent pouvoir être stockés en mémoire vive
  0. **Clustering initial:**
     Réaliser un clustering hiérarchique sur ces points
  0. **Choisir les représentants**
     Pour chaque cluster, choisir les points représentants: échantillon de points aussi dispersés que possible
     Déplacer les représentants d’une fraction vers le centroïde du cluster (par exemple 20%)
     Remarque: cette fraction est un **hyperparamètre**

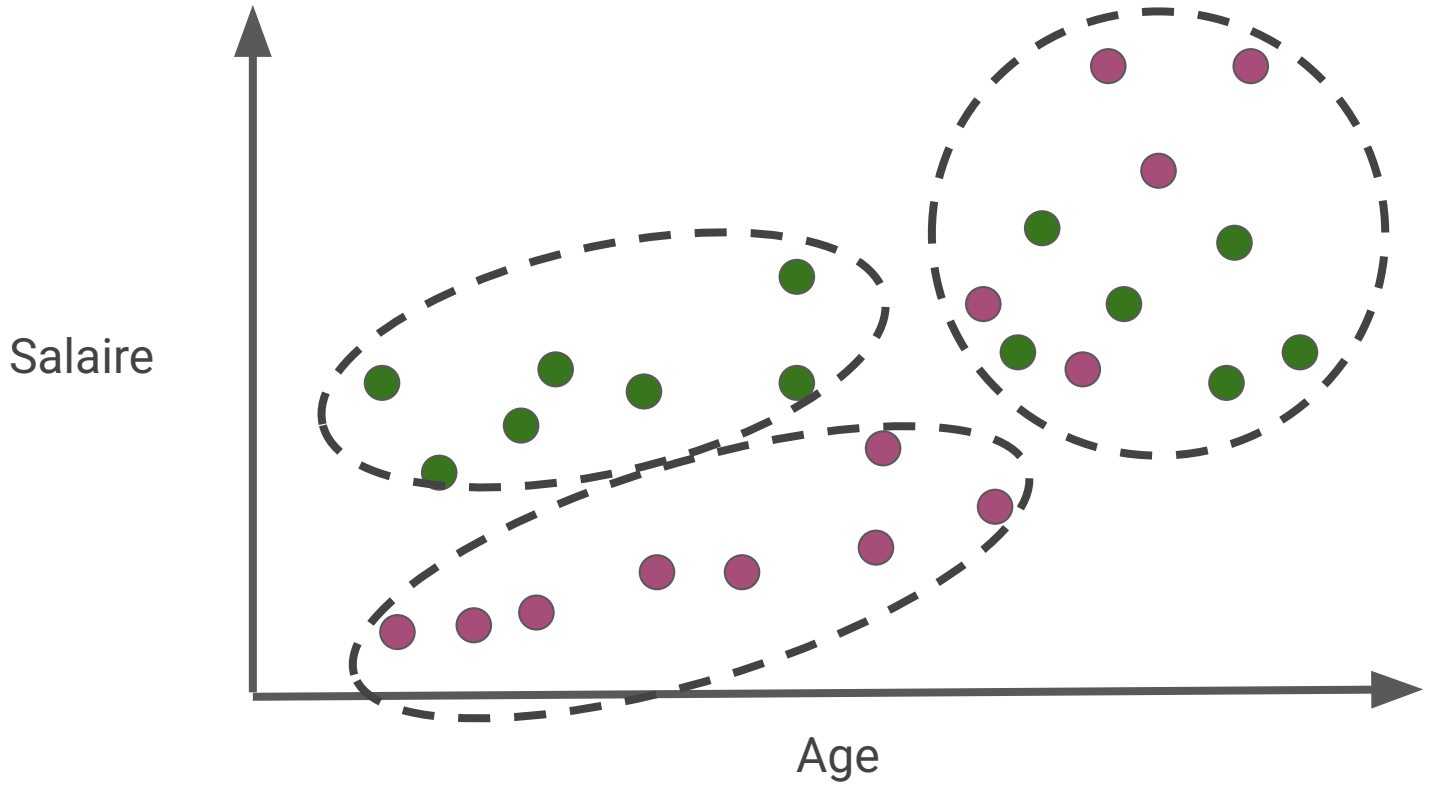
@ 2026 MKL.AI - Tous droits réservés.

CURE: clusters initiaux



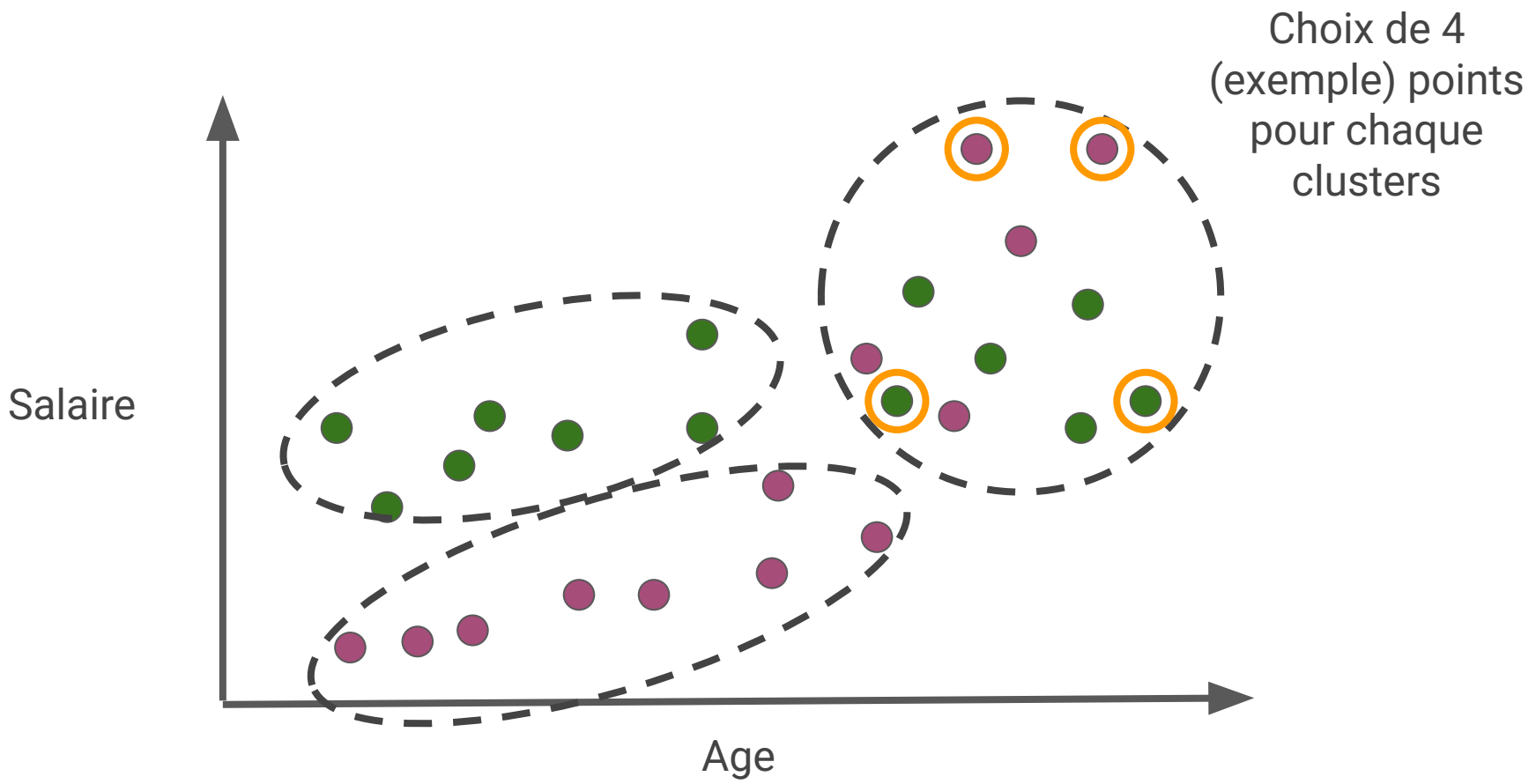
@ 2026 MKL.AI - Tous droits réservés.

CURE: clusters initiaux



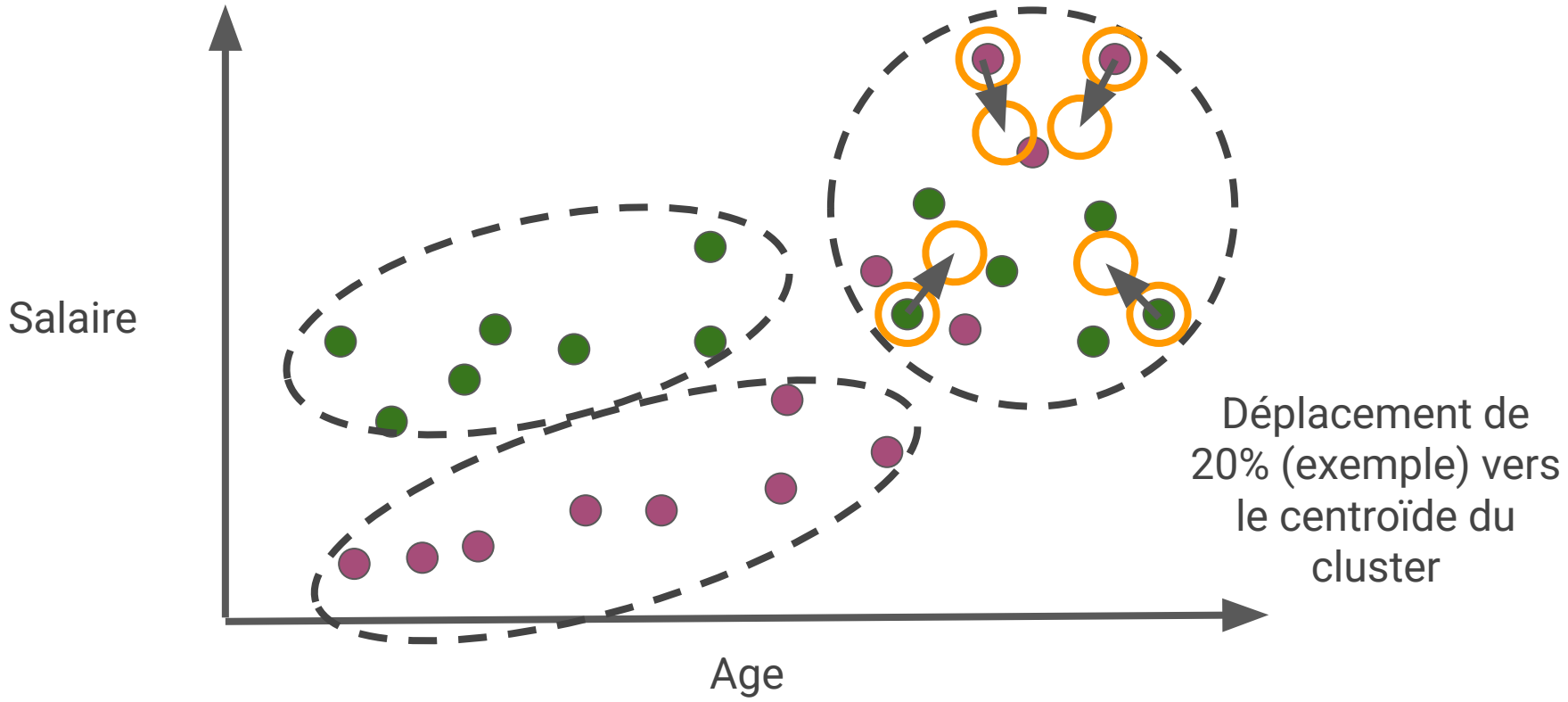
@ 2026 MKL.AI - Tous droits réservés.

CURE: représentants



@ 2026 MKL.AI - Tous droits réservés.

CURE: déplacement des représentants



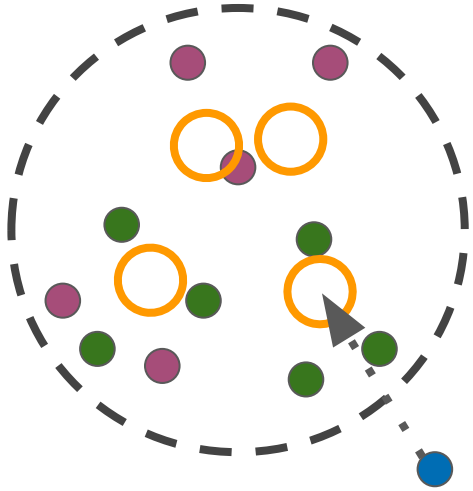
@ 2026 MKL.AI - Tous droits réservés.

Algorithme CURE (2/2)

- **Séquence 2**
  - **Repasser sur le jeu de données en entier et placer chaque point dans le cluster “le plus proche”**

    Ici, “le plus proche” signifie trouver le représentant le plus proche

    Assigner ensuite le point au cluster du représentant

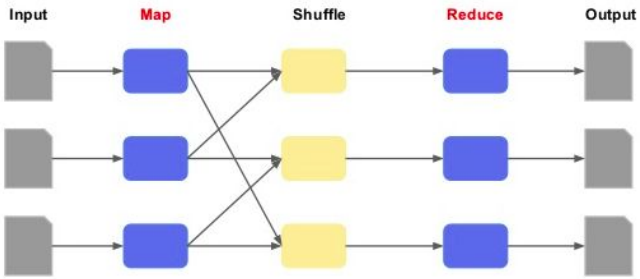


@ 2026 MKL.AI - Tous droits réservés.

Implémentation MapReduce (K-means)

MapReduce (rappels)

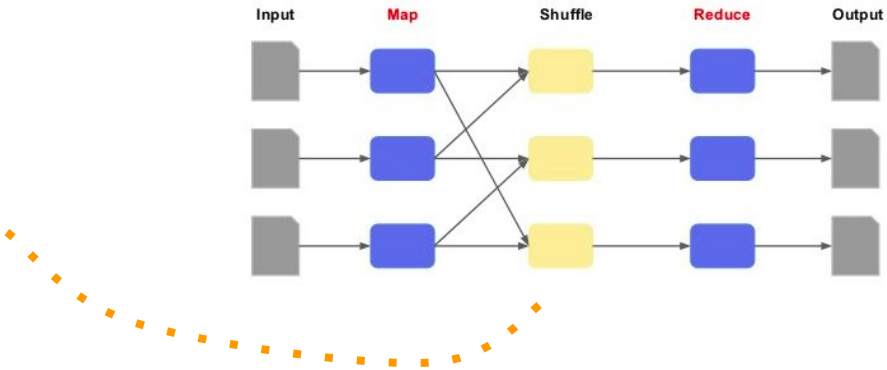
- Les tâches **Map** réalisent la conversion des entrées en **paires de clés-valeurs** (key-value pairs), non nécessairement uniques



@ 2026 MKL.AI - Tous droits réservés.

MapReduce (rappels)

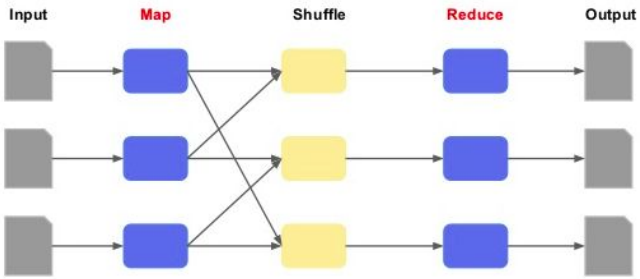
- Les tâches **Map** réalisent la conversion des entrées en **paires de clés-valeurs** (key-value pairs), non nécessairement uniques
- Les sorties des tâches Map sont **triées** par clé (shuffle/sort), et chaque clé est assignée à une tâche **Reduce**



@ 2026 MKL.AI - Tous droits réservés.

MapReduce (rappels)

- Les tâches **Map** réalisent la conversion des entrées en **paires de clés-valeurs** (key-value pairs), non nécessairement uniques
- Les sorties des tâches Map sont **triées** par clé (shuffle/sort), et chaque clé est assignée à une tâche **Reduce**
- Les tâches Reduce **combinent** les valeurs associées à une clé



@ 2026 MKL.AI - Tous droits réservés.

K-means (PySpark)

```python
def kmeans(rdd, n_centers, n_iter):
    # 0. Compute dataset bounding box
    min_ = rdd.reduce(lambda a, b: numpy.min((a, b), axis=0))
    max_ = rdd.reduce(lambda a, b: numpy.max((a, b), axis=0))

    # 1. Initialize cluster centroids
    n_dim = len(min_)
    centroids = numpy.random.uniform(min_, max_, (n_centers, n_dim))

    # 2. Repeat until convergence
    seqOp = lambda a, b: (a[0] + b, a[1] + 1)
    combOp = lambda a, b: (a[0] + b[0], a[1] + b[1])
    for i in range(n_iter):
        # 2.1 Compute label for each points
        labels = rdd.map(lambda x: (compute_label(x, centroids), x))

        # 2.2 Compute the new centroids by label and collect them in a dictionnary
        centmap = labels.aggregateByKey((numpy.zeros(n_dim), 0), seqOp, combOp)\
                        .mapValues(lambda x: x[0] / x[1])\
                        .collectAsMap()

        # 2.3 Update the centroids in the numpy matrix
        for i in range(n_centers):
            if i in centmap:
                centroids[i, :] = centmap[i]
            else:
                # No point were associated centroid labeled `i`
                # Generate a new random one
                centroids[i, :] = numpy.random.uniform(min_, max_, n_dim)

    # 3. Return the final centroids
    return centroids
```

@ 2026 MKL.AI - Tous droits réservés.

K-means (PySpark MLlib)

```python
from pyspark.mllib.clustering import KMeans, KMeansModel

clusters = KMeans.train(rdd, N_CENTERS, maxIterations=20, initializationMode="random")
```

@ 2026 MKL.AI - Tous droits réservés.

Apprentissage automatique avec MLlib

APACHE Spark MLlib

Download | Libraries | Documentation | Examples | Community | Developers | Apache Software Foundation

MLlib is Apache Spark's scalable machine learning library.

Ease of Use

Usable in Java, Scala, Python, and R.

MLlib fits into Spark's APIs and interoperates with NumPy in Python (as of Spark 0.9) and R libraries (as of Spark 1.5). You can use any Hadoop data source (e.g. HDFS, HBase, or local files), making it easy to plug into Hadoop workflows.

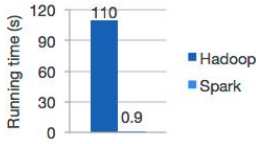
```
data = spark.read.format("libsvm")\
  .load("hdfs://...")

model = KMeans(k=10).fit(data)
```

Calling MLlib in Python

Performance

High-quality algorithms, 100x faster than MapReduce.

Spark excels at iterative computation, enabling MLlib to run fast. At the same time, we care about algorithmic performance: MLlib contains high-quality algorithms that leverage iteration, and can yield better results than the one-pass approximations sometimes used on MapReduce.



Logistic regression in Hadoop and Spark

Latest News

Spark 2.4.5 released (Feb 08, 2020)

Preview release of Spark 3.0 (Dec 23, 2019)

Preview release of Spark 3.0 (Nov 06, 2019)

Spark 2.3.4 released (Sep 09, 2019)

Archive

APACHE EVENTS LEARN MORE

Download Spark

Built-in Libraries:

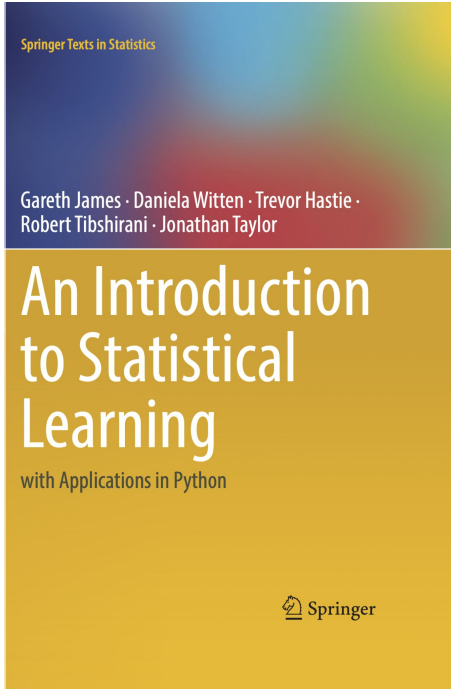
SQL and DataFrames
Spark Streaming
MLlib (machine learning)
GraphX (graph)

Third-Party Projects

@ 2026 MKL.AI - Tous droits réservés.

Lectures et références

Lectures

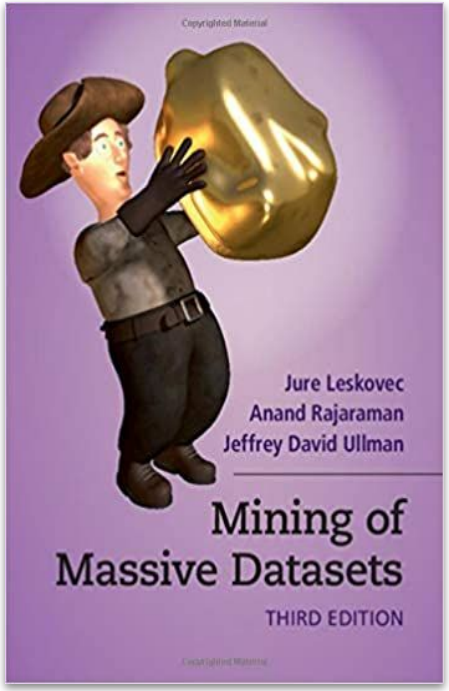


- Introduction to Statistical Learning with Applications in Python (2023)

  → 12.4 Clustering Methods

@ 2026 MKL.AI - Tous droits réservés.

Lectures



- Jure Leskovec, Anand Rajaraman, and Jeffrey D. Ullman, “Mining of Massive Datasets”

  → 7 Clustering p. 253-292

@ 2026 MKL.AI - Tous droits réservés.

Références

[1] Jure Leskovec, Anand Rajaraman, and Jeffrey D. Ullman, “Mining of Massive Datasets”

[2] Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani, “An Introduction to Statistical Learning with Applications in R”

[3] Introduction to PySpark - Tackling big data problems with a cluster computer, Calcul Québec

@ 2026 MKL.AI - Tous droits réservés.